Vetra: Offline AI-Based Health Monitoring and Early Disease Diagnosis for Farm Animals

Aayush Kumar
Department of [Department Name]
[Institution Name]
[City], India
[email address]

Abstract—Livestock disease is often noticed only once an animal is visibly ill, by which point treatment is costlier and infection may already have spread. This paper presents Vetra, a software system for continuous health monitoring and early diagnosis of farm animals that runs fully offline. Physiological and behavioural signals—heart rate, body temperature, respiratory rate, SpO_2, activity, rumination and gait—are grouped into one-hour windows, converted into 69 features, and classified into eight conditions by a compact neural network alongside rule-based clinical checks. An alert policy turns per-window predictions into a small number of actionable alerts, and records are stored locally and synchronised to a cloud dashboard when connectivity is available. Evaluated on 48 animals never seen during training, the neural network achieves a macro-F1 score of 0.981 with a model size of 12.2 KB, and the alert policy reduces alert volume by 99.2% while detecting all 35 sick animals and raising no alerts for the 13 healthy ones. Because the training data is simulated from published veterinary reference ranges, these results validate the pipeline rather than clinical performance.

Index Terms—precision livestock farming, animal health monitoring, disease diagnosis, edge AI, TensorFlow Lite, alert fatigue, offline inference

I. Introduction

Livestock is a major source of income for smallholder farmers in India, and an undetected illness in a single animal can mean lost milk yield, treatment costs or the spread of infection through a herd. In practice, disease is usually noticed by eye, once an animal stops eating or moving normally—well after the physiological signs first appeared.

Continuous monitoring can shift detection earlier. Changes in temperature, heart rate, breathing and rumination often precede visible symptoms [1], and precision livestock farming has made the case for using such signals to manage individual animals rather than whole herds [2]. Two practical obstacles remain for rural deployment. First, internet connectivity is intermittent, so a system that sends data to a server for diagnosis stops working when the link drops. Second, a system that raises an alert for every abnormal reading quickly overwhelms the farmer and is ignored.

Vetra addresses both. Every diagnosis is made locally with a model small enough for low-cost devices, and the cloud is used only to store and display records. A dedicated alert policy is treated as a core component, not an afterthought. The project is implemented entirely in software; animal vital signs are generated by a simulator built on published veterinary reference values.

The objectives of the project are:

1) A data acquisition and processing pipeline for animal vital signs.
2) Machine learning models that predict disease from those signals.
3) A lightweight model that runs without internet access.
4) Intelligent alert generation that limits false and repeated alerts.
5) Health record storage with cloud synchronisation.
6) A dashboard for farmers and veterinarians.

II. Related Work

Wearable sensors for livestock now measure temperature, activity, rumination and location, and have been reviewed extensively as tools for early disease detection [1]. Precision livestock farming frames these sensors as part of a control loop in which each animal is monitored and managed individually [2]. Edge computing moves computation close to the data source, reducing dependence on network links and lowering latency [3]; quantisation techniques make neural network inference practical on such constrained hardware [4].

Vetra combines these ideas into a single pipeline aimed at intermittently connected farms, and adds an explicit, measured alert policy, since alert volume largely decides whether such a system is used in practice.

III. System Architecture

Fig. 1 shows the system. It has three tiers: a local gateway that performs all diagnosis, a cloud service that stores records, and a web dashboard.

A. Local gateway

The gateway receives readings over MQTT [5] or REST, validates them against plausible physiological ranges, and stores them in a local SQLite database. For each animal it keeps a rolling 60-minute window; every 30 minutes it computes features, runs the model and the clinical rules,

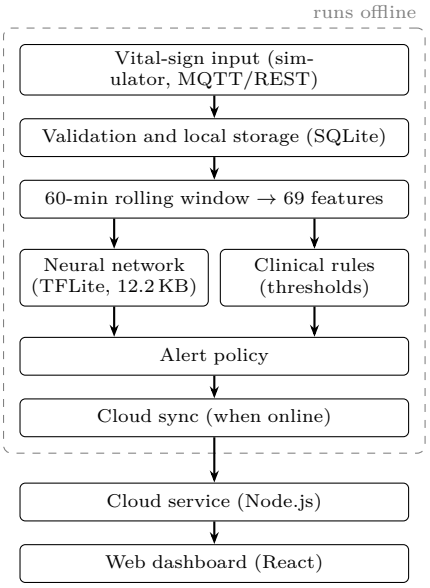


Fig. 1. Vetra architecture. Everything inside the dashed box runs without an internet connection.

and passes the result to the alert policy. If the network is unavailable, monitoring continues and records accumulate locally.

B. Cloud service and dashboard

A Node.js service receives synchronised animals, telemetry, diagnoses and alerts, stores them as health records, and pushes updates to a React dashboard using server-sent events. The dashboard shows the herd, per-animal vital-sign charts and open alerts, which a user can acknowledge.

C. Synchronisation

Each local table carries a nullable synced_at column instead of a separate outbox. Rows are marked synced only after the server acknowledges them, so an interrupted upload is simply retried and no record is lost. Tables are uploaded in dependency order—animals, telemetry, diagnoses, alerts.

IV. Methodology

A. Data generation

No public dataset of continuous, labelled, multi-channel vital signs for Indian livestock was available. Vetra therefore simulates a herd of 240 animals over five days at one reading per minute, covering cattle, buffalo, goats and sheep. Resting ranges for each species are taken from standard veterinary references [6], [7]; for example, cattle are modelled with a heart rate of 48–84 bpm and a rectal temperature of 38.0–39.3 °C. Heat load is modelled with the temperature–humidity index (THI) [8]:

$$\mathrm{THI} = 0.8\,T + \frac{RH}{100}\,(T - 14.4) + 46.4, \qquad (1)$$

TABLE I
Simulated Conditions and Their Distinguishing Signs

Condition	Distinguishing sign
Cardiac disorder	Heart rate stays high at rest; irregular rhythm
Respiratory disease	Faster breathing with falling SpO_2
Obesity	Chronic inactivity; no fever; normal gait
Diabetes	High variance in activity rather than a shifted mean
Heat stress	Panting under high THI; SpO_2 near normal
Infectious disease	Fever with preserved SpO_2
Mobility disorder	Irregular gait with normal temperature

where T is air temperature in °C and RH is relative humidity in percent.

Each disease is encoded as the change it produces in each signal at full severity (Table I). Acute conditions ramp up from an onset time over 6–36 hours; chronic ones (obesity, diabetes) are present from the start. A window is labelled diseased only once mean severity exceeds 0.15, so the model is not trained to find illness in noise.

B. Feature extraction

Readings are grouped into 60-minute windows with 50% overlap. Each window yields 69 features of two kinds. Species-normalised absolute values detect conditions already present when monitoring begins, such as obesity. Deviations from the animal's own trailing 24-hour baseline detect acute change, such as fever; the baseline is shifted by one sample so a window never contributes to the baseline it is compared against. The same feature function is used during training and on the gateway, and a test checks that both paths produce identical output.

C. Train–test split

Because overlapping windows from one animal are nearly identical, a random split would place almost the same rows in both training and test sets and inflate every metric. The data is therefore split by animal: 20% of animals are held out for testing and 15% of the remainder for validation, with no animal in more than one set. All results are reported on the 48 held-out test animals (11,472 windows).

D. Models

Three classical baselines were trained with scikit-learn [9]: multinomial logistic regression, a random forest [10] with 300 trees, and histogram gradient boosting [11]. The deployed model is a compact neural network built in TensorFlow [12]: a normalisation layer, dense layers of 64 and 32 ReLU units with dropout of 0.2 and 0.1, and an eight-class softmax output. It is trained with Adam [13] (learning rate 10^{-3}, batch size 256, up to 80 epochs) with class weighting and early stopping on validation loss.

TABLE II
Diagnosis Performance on Held-Out Animals

Metric	Logistic regression	Neural network
Macro-F1	0.986	0.981
Screening recall	0.985	0.977
False-alarm rate	0.008	0.009
Model size	7 KB	12.2 KB

E. Edge model export

The network is converted to TensorFlow Lite in three variants: float32, dynamic-range int8 and full int8 [4]. Each variant is compared with the float model on the test set, and any variant that disagrees on more than 1% of predictions is rejected automatically. The normalisation layer is kept inside the exported model, so no separate scaler file has to be kept in step with it.

F. Clinical rules

Rule-based checks run alongside the model and fire on textbook thresholds regardless of its prediction—for example fever, resting tachycardia, low SpO_2, a sharp drop in rumination against the animal's baseline, and THI above 78, 84 or 88. The rules give the veterinarian a readable reason ("SpO_2 88%, respiratory rate 64/min") rather than a bare probability, and ensure a critical sign is never missed because of a model error.

G. Alert policy

Raising an alert for every abnormal window would produce about 26 alerts per animal per day. The policy instead requires:

- Confirmation: two matching windows out of the last four, each with confidence of at least 0.60.
- Hysteresis: three consecutive healthy windows before an alert is resolved.
- Cooldown: six hours before the same condition can re-alert.
- Deduplication: at most one open alert per animal per condition.

A critical clinical rule bypasses confirmation and alerts immediately.

V. Results

A. Diagnosis

Table II compares the best classical baseline, logistic regression, with the deployed neural network on the held-out animals, and Table III gives the network's per-class scores. Diabetes has the lowest recall (0.921); most of its errors are windows classified as healthy, which is expected since it is defined by activity variance rather than a clear shift in any one signal.

Since the aim is early detection, recall was also measured by disease severity (Table IV). Early-stage cases are the hardest, at 88.4% recall, with recall above 96% at later stages.

TABLE III
Neural Network Per-Class Performance

Class	Precision	Recall	F1	Windows
Healthy	0.972	0.991	0.982	5,177
Cardiac disorder	0.982	0.992	0.987	611
Respiratory disease	0.946	1.000	0.972	369
Obesity	0.997	0.993	0.995	1,434
Diabetes	0.993	0.921	0.955	1,673
Heat stress	0.988	0.975	0.981	670
Infectious disease	0.990	0.986	0.988	707
Mobility disorder	0.982	0.999	0.991	831

TABLE IV
Neural Network Recall by Disease Severity

Severity band	Severity range	Recall
Early	0.15–0.35	0.884
Mild	0.35–0.55	0.993
Marked	0.55–0.75	0.967
Severe	0.75–1.00	0.984

B. Edge deployment

Table V lists the exported variants. Full-int8 quantisation agreed with the float model on only 46% of predictions and fell to 0.08 macro-F1, because the raw features span very different scales (from about 0.9 to several thousand) and an 8-bit input cannot represent both. The agreement check rejected it automatically, and the dynamic-range int8 model was selected. The full streaming pipeline (validation, windowing, feature extraction and inference) processes about 5,000 readings per second on a single core of a development laptop.

C. Alerts

The alert policy was evaluated on the 48 held-out animals over 240 animal-days (Table VI). It reduces alert volume by 99.2% without missing any sick animal, and the median delay from disease onset to alert is 1.5 hours.

D. Testing

The system is covered by 70 Python tests and 20 Node.js API tests, all passing. They include checks that no animal appears in two data splits, that training and streaming features are identical, and that the alert policy, rules and local store behave as specified.

VI. Discussion and Limitations

The main limitation is that the data is simulated. The results therefore show that the pipeline is sound—the split does not leak, the features carry the signal, the model fits in 12.2 KB and runs offline, and the alert policy suppresses noise without losing sick animals—but they do not measure clinical accuracy. That a logistic regression scores 0.986 suggests the simulated disease signatures are cleaner than real presentations; on real data all metrics would be expected to fall, with non-linear models likely pulling ahead.

TABLE V
TensorFlow Lite Variants

Variant	Size	Agreement	Latency	Status
float32	29.9 KB	1.0000	0.0066 ms	–
Dynamic int8	12.2 KB	0.9997	0.0064 ms	Selected
Full int8	13.0 KB	0.4599	0.0076 ms	Rejected

TABLE VI
Alert Policy Evaluation (48 Animals, 240 Animal-Days)

Measure	Every abnormal window	With policy
Alerts raised	6,200	52
Alerts per animal-day	25.83	0.22
Sick animals detected	35/35	35/35
Healthy animals alerted	–	0/13
Median time from onset	–	1.5 h

Real animals also present with multiple simultaneous conditions, missing sensor readings and diseases outside the eight modelled classes. The design limits the cost of moving to real data: only the data generator needs to be replaced by a loader producing the same columns.

The current security model is minimal: a shared API key protects synchronisation, and the dashboard has no user accounts. A production deployment would need per-gateway credentials, authenticated sessions and encrypted connections.

VII. Conclusion and Future Work

Vetra demonstrates an end-to-end, offline-capable pipeline for livestock health monitoring: from vital signs through feature extraction, a 12.2 KB diagnostic model and clinical rules, to an alert policy that cuts alert volume by 99.2% while detecting every sick animal in the test herd.

Planned work includes:

- Symptom-selection interface: a form where the user selects an animal's symptoms or readings from drop-down lists and receives a diagnosis, confidence level and recommended action.
- Veterinarian feedback: recording confirmed diagnoses against each alert, building a real labelled dataset for retraining.
- Farmer notifications: SMS or WhatsApp alerts in local languages.
- Outbreak detection: flagging clusters of nearby animals that show infectious signs within a short period.

References

[1] S. Neethirajan, "Recent advances in wearable sensors for animal health management," Sens. Bio-Sens. Res., vol. 12, pp. 15–29, 2017.

[2] D. Berckmans, "General introduction to precision livestock farming," Anim. Front., vol. 7, no. 1, pp. 6–11, 2017.

[3] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," IEEE Internet Things J., vol. 3, no. 5, pp. 637–646, Oct. 2016.

[4] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2018, pp. 2704–2713.

[5] OASIS, "MQTT Version 3.1.1," OASIS Standard, Oct. 2014.

[6] MSD Veterinary Manual. Rahway, NJ, USA: Merck & Co. [Online]. Available: https://www.msdvetmanual.com

[7] P. G. G. Jackson and P. D. Cockcroft, Clinical Examination of Farm Animals. Oxford, U.K.: Blackwell Science, 2002.

[8] National Research Council, A Guide to Environmental Research on Animals. Washington, DC, USA: Nat. Acad. Sci., 1971.

[9] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[10] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5–32, 2001.

[11] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," Ann. Statist., vol. 29, no. 5, pp. 1189–1232, 2001.

[12] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in Proc. 12th USENIX Symp. Oper. Syst. Des. Implement. (OSDI), 2016, pp. 265–283.

[13] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. Int. Conf. Learn. Represent. (ICLR), 2015.